1

Why React?

If you're reading this book, you probably are already familiar with **React**. But if you're not, don't worry. I'll do my best to keep philosophical definitions to a minimum. However, this is a long book with a lot of content, so I feel that setting the tone is an appropriate first step. Our goal is to learn React and **React Native**, but it's also to build a scalable and adaptive architecture that can handle everything we want to build with React today and in the future. In other words, we want to create a foundation around React, with a set of additional tools and approaches that can withstand the test of time. This book will guide you through the process of using tools like routing, TypeScript typing, testing, and many more.

This chapter starts with a brief explanation of why React ex-

is able to handle many of the typical performance issues faced by web developers. Next, we'll go over the declarative philosophy of React and the level of abstraction that React program-

mers can expect to work with. Then, we'll touch on some of the major features of React. And finally, we will explore how we can set up a project to start to work with React.

Once you have a conceptual understanding of React and how it solves problems with UI development, you'll be better equipped to tackle the remainder of the book. This chapter will cover the following topics:

- What is React?
- What's new in React?
- Setting up a new React project

What is React?

I think the one-line description of React on its home page (**https://react.dev/**) is concise and accurate:

> *"A JavaScript library for building user interfaces."*

This is perfect because, as it turns out, this is all we want most of the time. I think the best part about this description is everything that it leaves out. It's not a mega-framework. It's not a full-stack solution that's going to handle everything, from the database to real-time updates over **WebSocket** connections.

We might not actually want most of these prepackaged solutions. If React isn't a framework, then what is it exactly?

React is just the view layer

React is generally thought of as the *view layer* in an application. Applications are typically divided into different layers, such as the view layer, the logic layer, and the data layer. React, in this context, primarily handles the view layer, which involves rendering and updating the UI based on changes in data and application state. React components change what the user sees. The following diagram illustrates where React fits in our frontend code:

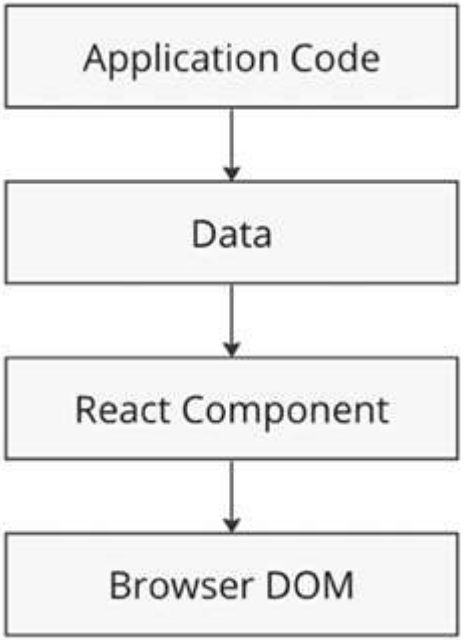

Figure 1.1: The layers of a React application

This is all there is to React – the core concept. Of course, there will be subtle variations to this theme as we make our way through the book, but the flow is more or less the same:

1. **Application logic**: Start with some application logic that generates data.
2. **Rendering data to the UI**: The next step is to render this data to the UI.
3. **React component**: To accomplish this, you pass the data to a React component.
4. **Component's role**: The React component takes on the responsibility of getting the HTML onto the page.

You may wonder what the big deal is; React appears to be yet another rendering technology. We'll touch on some of the key areas where React can simplify application development in the remaining sections of the chapter.

Simplicity is good

React doesn't have many moving parts to learn about and understand. While React boasts a relatively simple API, it's important to note that beneath the surface, React operates with a degree of complexity. Throughout this book, we will delve into these internal workings, exploring various aspects of React's

architecture and mechanisms to provide you with a comprehensive understanding. The advantage of having a small API to work with is that you can spend more time familiarizing yourself with it, experimenting with it, and so on. The opposite is true of large frameworks, where all of your time is devoted to figuring out how everything works. The following diagram gives you a rough idea of the APIs that we have to think about when programming with React:

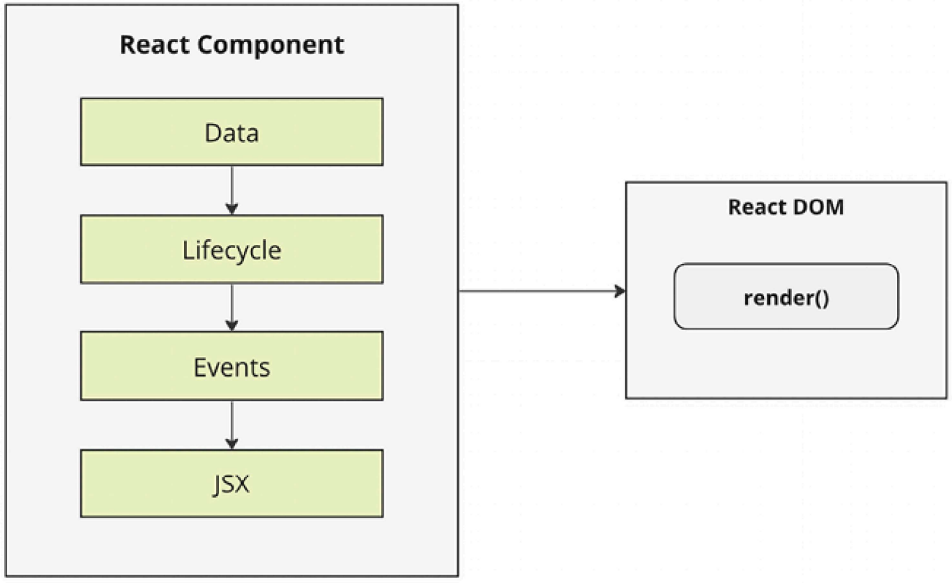

Figure 1.2: The simplicity of the React API

React is divided into two major APIs:

- **The React Component API**: These are the parts of the page that are rendered by the **React DOM**.
- **React DOM**: This is the API that's used to perform the rendering on a web page.

Within a React component, we have the following areas to think about:

- **Data**: This is data that comes from somewhere (the component doesn't care where) and is rendered by the component.
- **Lifecycle**: For example, one phase of the lifecycle is when the component is about to be rendered. Within a React component, methods or hooks respond to the component's entering and exiting phases of the React rendering process as they happen over time.
- **Events**: These are the code that we write to respond to user interactions.
- **JSX**: This is the syntax commonly used for describing UI structures in React components. Even though JSX is closely associated with React, it can also be used alongside other **JavaScript** frameworks and libraries.

Don't fixate on what these different areas of the React API represent just yet. The takeaway here is that React, by nature, is simple. Just look at how little there is to figure out! This means

that we don't have to spend a ton of time going through API details here. Instead, once you pick up on the basics, we can spend more time on nuanced React usage patterns that fit in nicely with declarative UI structures.

Declarative UI structures

React newcomers have a hard time getting to grips with the idea that components mix in markup with their JavaScript in order to declare UI structures. If you've looked at React examples and had the same adverse reaction, don't worry. Initially, we can be skeptical of this approach, and I think the reason is that we've been conditioned for decades by the *separation of concerns* principle. This principle states that different concerns, such as logic and presentation, should be separate from one another. Now, whenever we see things combined, we automatically assume that this is bad and shouldn't happen.

The syntax used by React components is called **JSX** (short for **JavaScript XML**, also known as **JavaScript Syntax Extension**). A component renders content by returning some JSX. The JSX itself is usually HTML markup, mixed with custom tags for React components. The specifics don't matter at this point; we'll go into detail in the coming chapters.

What's groundbreaking about the declarative JSX approach is that we don't have to manually perform intricate operations to change the content of a component. Instead, we describe how the UI should look in different states, and React efficiently updates the actual DOM to match. As a result, React UIs become easier and more efficient to work with, resulting in better performance.

For example, think about using something such as jQuery to build your application. You have a page with some content on it, and you want to add a class to a paragraph when a button is clicked:

```
$(document).ready(function() {
  $('#my-button').click(function() {
    $('#my-paragraph').addClass('highlight');
  });
});
```

Performing these steps is easy enough. This is called imperative programming, and it's problematic for UI development. The problem with imperative programming in UI development is that it can lead to code that is difficult to maintain and modify. This is because imperative code is often tightly coupled, meaning that changes to one part of the code can have unintended consequences elsewhere. Additionally, imperative code

can be difficult to reason about, as it can be hard to understand the flow of control and the state of an application at any given time. While this example of changing the class of an element is simple, real applications tend to involve more than three or four steps to make something happen.

React components don't require you to execute steps in an imperative way. This is why JSX is central to React components. The XML-style syntax makes it easy to describe what the UI should look like – that is, what are the HTML elements that component is going to render?

```
export const App = () => {
  const [isHighlighted, setIsHighlighted] = useState(false);
  return (
    <div>
      <button onClick={() => setIsHighlighted(true)}>Add Class</button>
      <p className={isHighlighted && "highlight"}>This is paragraph</p>
    </div>
  );
};
```

In this example, we're not just writing the imperative procedure that the browser should execute. This is more like an instruction, where we say how the UI should look and what user interaction should happen on it. This is called declarative programming and is very well suited for UI development. Once

you've declared your UI structure, you need to specify how it changes over time.

Data changes over time

Another area that's difficult for React newcomers to grasp is the idea that JSX is like a static string, representing a chunk of rendered output. This is where data and the passage of time come into play. React components rely on data being passed into them. This data represents the dynamic parts of the UI – for example, a UI element that's rendered based on a Boolean value could change the next time the component is rendered. Here's a diagram illustrating the idea:

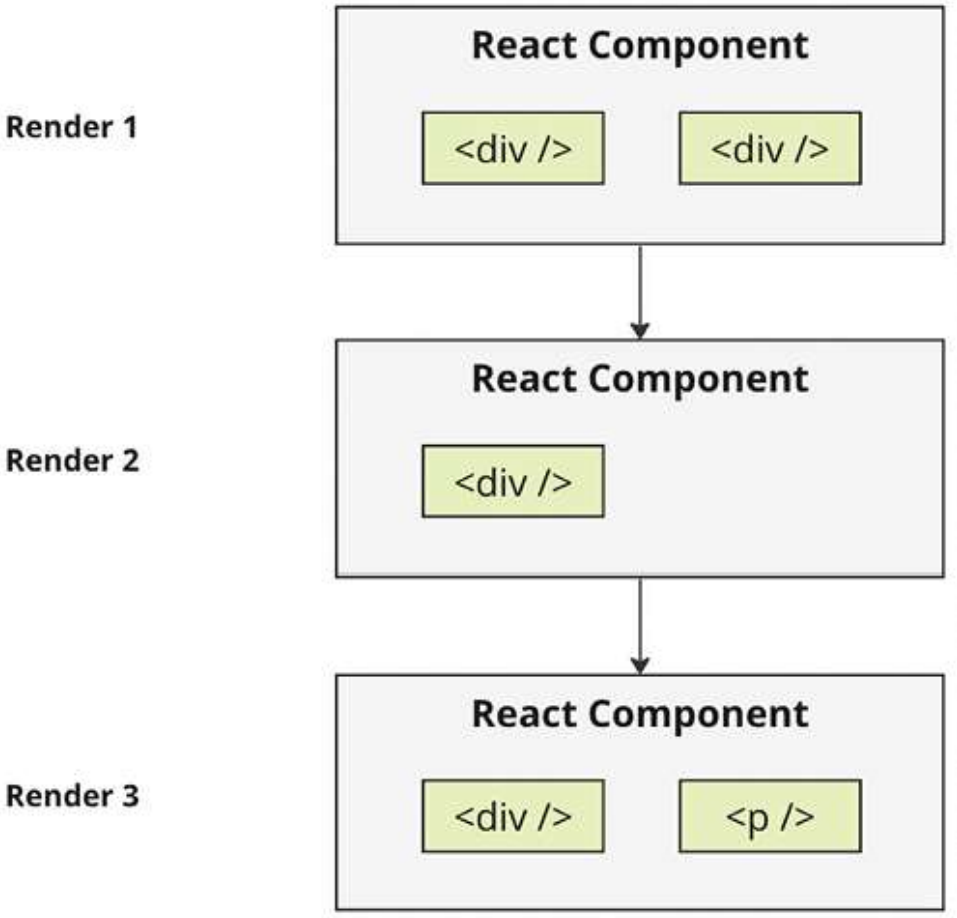

Figure 1.3: React components changing over time

Each time the React component is rendered, it's like taking a snapshot of the JSX at that exact moment in time. As your application moves forward through time, you have an ordered collection of rendered UI components. In addition to declaratively describing what a UI should be, re-rendering the same JSX content makes things much easier for developers. The chal-

lenge is making sure that React can handle the performance demands of this approach.

Performance matters

Using React to build UIs means that we can declare the structure of the UI with JSX. This is less error-prone than the imperative approach of assembling the UI piece by piece. However, the declarative approach does present a challenge with performance.

For example, having a declarative UI structure is fine for the initial rendering because there's nothing on the page yet. So the React renderer can look at the structure declared in JSX and render it in the DOM browser.

The Document Object Model (DOM) represents HTML in the browser after it has been rendered. The DOM API is how JavaScript is able to change content on a page.

This concept is illustrated in the following diagram:

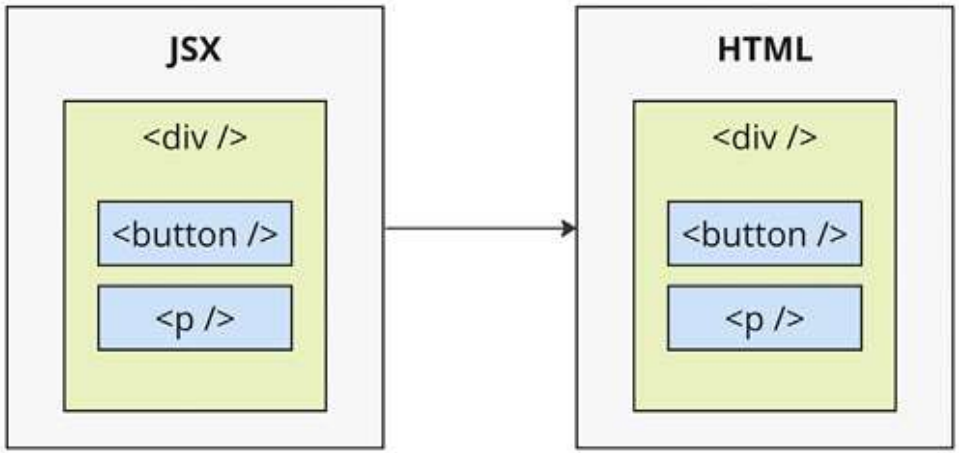

Figure 1.4: How JSX syntax translates to HTML in the browser DOM

On the initial render, React components and their JSX are no different from other template libraries. For instance, there is a templating library called **Handlebars** used for server-side rendering, which will render a template to HTML markup as a string that is then inserted into the browser DOM. Where React is different from libraries such as Handlebars is that React can accommodate when data changes and we need to re-render the component, whereas Handlebars will just rebuild the entire HTML string, the same way it did on the initial render. Since this is problematic for performance, we often end up implementing imperative workarounds that manually update tiny bits of the DOM. We end up with a tangled mess of declar-

ative templates and imperative code to handle the dynamic aspects of the UI.

We don't do this in React. This is what sets React apart from other view libraries. Components are declarative for the initial render, and they stay this way even as they're re-rendered. It's what React does under the hood that makes re-rendering declarative UI structures possible.

In React, however, when we create a component, we describe what it should look like clearly and straightforwardly. Even as we update our components, React handles the changes smoothly behind the scenes. In other words, components are declarative for the initial render, and they stay this way even as they're re-rendered. This is possible because React employs the virtual DOM, which is used to keep a representation of the real DOM elements in memory. It does this so that each time we re-render a component, it can compare the new content to the content that's already displayed on the page. Based on the difference, the virtual DOM can execute the imperative steps necessary to make the changes. So not only do we get to keep our declarative code when we need to update the UI but React will also make sure that it's done in a performant way. Here's what this process looks like:

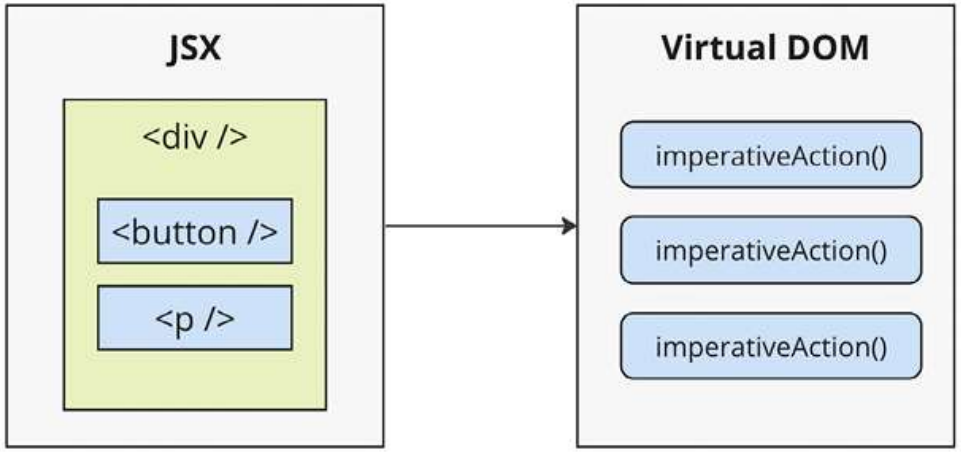

Figure 1.5: React transpiles JSX syntax into imperative DOM API calls

When you read about React, you'll often see words such as **diffing** and **patching**. Diffing means comparing *old content* (the previous state of the UI) with *new content* (the updated state) to identify the differences, much like comparing two versions of a document to see what's changed. Patching means executing the necessary DOM operations to render the new content, ensuring that only the specific changes are made, which is crucial for performance.

As with any other JavaScript library, React is constrained by the *run-to-completion* nature of the main thread. For example,

if the React virtual DOM logic is busy diffing content and patching the real DOM, the browser can't respond to user input such as clicks or interactions.

As you'll see in the next section of this chapter, changes were made to the internal rendering algorithms in React to mitigate these performance pitfalls. With performance concerns addressed, we need to make sure that we're confident that React is flexible enough to adapt to the different platforms that we might want to deploy our apps to in the future.

The right level of abstraction

Another topic I want to cover at a high level before we dive into React code is **abstraction**.

In the preceding section, you saw how JSX syntax translates to low-level operations that update our UI. A better way to look at how React translates our declarative UI components is via the fact that we don't necessarily care what the render target is. The render target happens to be the browser DOM with React, but, as we will see, it isn't restricted to the browser DOM.

React has the potential to be used for any UI we want to create, on any conceivable device. We're only just starting to see this with React Native, but the possibilities are endless. I would not be surprised if *React Toast*, which is totally not a thing, sud-

denly becomes relevant, where React targets toasters that can singe the rendered output of JSX onto bread. React's abstraction level strikes a balance that allows for versatility and adaptability while maintaining a practical and efficient approach to UI development.

The following diagram gives you an idea of how React can target more than just the browser:

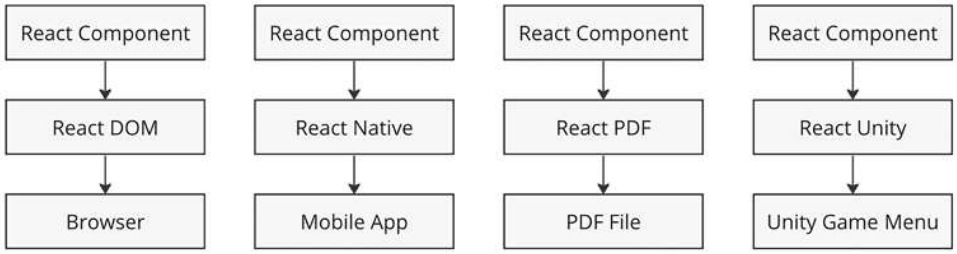

Figure 1.6: React abstracts the target rendering environment from the components that we implement

From left to right, we have **React DOM**, **React Native**, **React PDF**, and **React Unity**. All of these React Renderer libraries accept the React component and return a platform-specific result. As you can see, to target something new, the same pattern applies:

- Implement components specific to the target.

- Implement a React renderer that can perform the platform-specific operations under the hood.

This is, obviously, an oversimplification of what's actually implemented for any given React environment. But the details aren't so important to us. What's important is that we can use our React knowledge to focus on describing the structure of our UI on any platform.

Now that you understand the role of abstractions in React, let's see what's new in React.

What's new in React?

React is a continuously evolving library in the ever-changing web development landscape. As you embark on your journey to learn and master React, it's important to understand the evolution of the library and its updates over time.

One of the advantages of React is that its core API has remained relatively stable in recent years. This provides a sense of continuity and allows developers to leverage their knowledge from previous versions. The conceptual foundation of React has remained intact, meaning that the skills acquired three or five years ago can still be applied today. Let's take a step back and trace the history of React from its early versions

to the recent ones. From **React 0.x** to **React 18**, numerous pivotal changes and enhancements have been made as follows:

- **React 0.14**: In this version, the introduction of functional components allowed developers to utilize functions as components, simplifying the creation of basic UI elements. At that time, no one knew that now we would write only functional components and almost completely abandon class-based components.
- **React 15**: With a new versioning scheme, the next update of React 15 brought a complete overhaul of the internal architecture, resulting in improved performance and stability.
- **React 16**: This version, however, stands as one of the most notable releases in React's history. It introduced hooks, a revolutionary concept that enables developers to use state and other React features without the need for class components. Hooks make code simpler and more readable, transforming the way developers write components. We will explore a lot of hooks in this book. Additionally, React 16 introduced **Fiber**, a new reconciliation mechanism that significantly improved performance, especially when dealing with animations and complex UI structures.
- **React 17**: This version focused on updating and maintaining compatibility with previous versions. It introduced a new JSX transform system.

- **React 18**: This release continues the trajectory of improvement and emphasizes performance enhancements and additional features, such as the automatic batching of renders, state transitions, server components, and streaming server-side rendering. Most of the important updates related to performance will be explored in *Chapter 12*, *High-Performance State Updates*. More details about server rendering will be covered in *Chapter 14*, *Server Rendering and Static Site Generation with React Frameworks*.
- **React 19**: Introduces several major features and improvements. The **React Compiler** is a new compiler that enables automatic memoization and optimizes re-rendering, eliminating the need for manual `useMemo`, `useCallback`, and memo optimizations. Enhanced **Hooks** like `use` (promise) for data fetching, `useFormStatus()` and `useFormState()` for form handling, and `useOptimistic()` for optimistic UI simplify common tasks. React 19 also brings simplified APIs, such as ref becoming a regular prop, React.lazy being replaced, and Context.Provider becoming just Context. Asynchronous rendering allows fetching data asynchronously during rendering without blocking the UI, while error handling improvements provide better mechanisms to diagnose and fix issues in applications.

React's stability and compatibility make it a reliable library for long-term use, while the continuous updates ensure that it re-

mains at the forefront of web and mobile development. Throughout this book, all examples will utilize the latest React API, ensuring that they remain functional and relevant in future versions.

Now that we have explored the evolution and updates in React, we can delve deeper into React and examine how to get set up with the new React project.

Setting up a new React project

There are several ways to create a React project when you are getting started. In this section, we will explore three common approaches:

- Using web bundlers
- Using frameworks
- Using online code editors

To start developing and previewing your React applications, you will first need to have **Node.js** installed on your computer. Node.js is a runtime environment for executing JavaScript code.

Let's dive into each approach in the following subsections.

Using web bundlers

Using a web bundler is an efficient way to create React projects, especially if you are building a **single-page application (SPA)**. For all of the examples in this book, we will use **Vite** as our web bundler. Vite is known for its remarkable speed and ease of setup and use.

To set up your project using Vite, you will need to take the following steps:

1. Ensure that you have Node.js installed on your computer by visiting the official *Node.js* website (https://nodejs.org/) and downloading the appropriate version for your operating system.
2. Open your terminal or command prompt and navigate to the directory where you want to create your project:

```
mkdir react-projects
cd react-projects
```

3. Run the following command to create a new React project with Vite:

```
npm create vite@latest my-react-app -- --template react
```

This command creates a new directory called `my-react-app` and sets up a React project using the Vite template.

4. Once the project is created, your terminal should look like this:

```
Scaffolding project in react-projects/my-react-app...
Done. Now run:
  cd my-react-app
  npm install
     npm run dev
```

5. Navigate into the project directory and install dependencies. The result in the terminal should look like:

```
added 279 packages, and audited 280 packages in 21s
103 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
```

Finally, start the development server by running the following command: `npm run dev`

This command launches the development server, and you can view your React application by opening your browser and visiting `http://localhost:3000`.

By now, you will have successfully set up your React project using Vite as the web bundler. For more information about Vite and its possible configurations, visit the official website at https://vitejs.dev/.

Using frameworks

For real-world and commercial projects, it is recommended to use frameworks built on top of React. These frameworks provide additional features out of the box, such as routing and asset management (images, SVG files, fonts, etc.). They also guide you in organizing your project structure effectively, as frameworks often enforce specific file organization rules. Some popular React frameworks include **Next.js**, **Gatsby**, and **Remix**.

In *Chapter 13*, *Server-Side Rendering*, we will explore setting up Next.js and some differences between that and using a plain web bundler.

Online code editors

Online code editors combine the advantages of web bundlers and frameworks but allow you to set up your React development environment in the cloud or right inside of the browser. This eliminates the need to install anything on your machine

and lets you write and explore React code directly in your browser.

While there are various online code editors available, some of the most popular options include **CodeSandbox**, **StackBlitz**, and **Replit**. These platforms provide a user-friendly interface and allow you to create, share, and collaborate on React projects without any local setup.

To get started with an online code editor, you don't even need an account. Simply follow this link on your browser: `https://react.new`. In a few seconds, you will see that CodeSandbox is ready to work with a template project, and a live preview of the editor is available directly in the browser tab. If you want to save your changes, then you need to create an account.

Using online code editors is a convenient way to learn and experiment with React, especially if you prefer a browser-based development environment.

In this section, we explored different methods to set up your React project. Whether you choose web bundlers, frameworks, or online code editors, each approach offers its unique advantages. Select the method that you prefer and suits your project requirements. Now, we are ready to dive into the world of React development!

Summary

In this chapter, you were introduced to React comprehensively so that you have an idea of what it is and the necessary aspects of it, setting the tone for the rest of the book. React is a library with a small API used to build UIs. Then, you were introduced to some of the key concepts of React. We discussed the fact that React is simple because it doesn't have a lot of moving parts.

Afterward, we explored the declarative nature of React components and JSX. Following that, you learned that React enables effective performance by writing declarative code that can be re-rendered repeatedly.

You also gained insight into the idea of render targets and how React can easily become the UI tool of choice for various platforms. We then provided you with a brief overview of React's history and introduced the latest developments. Finally, we delved into how to set up a new React project and initiate the learning process.

That's sufficient introductory and conceptual content for now. As we progress through the book's journey, we'll revisit these concepts. Next, let's take a step back and nail down the basics, starting with rendering with JSX in the next chapter.

Join us on Discord!

Read this book alongside other users and the authors themselves. Ask questions, provide solutions to other readers, chat with the authors, and more. Scan the QR code or visit the link to join the community.

https://packt.link/ReactAndReactNative5e